Neural Differential Equations + Stochastic Calculus + Finance — Coding Workbook Companion (Full)

This companion mirrors the main roadmap's half-month structure. Each section explains **why** the task matters (linking to the Reading), gives **ordered steps**, **runnable code**, and the **expected outcome**. All examples use **Python** (NumPy/Matplotlib/PyTorch) and are Jupyter/Colab-friendly.

> Tip: keep a `utils.py` with seeding/plot helpers and import it across notebooks.

🟩 Oct 12 – Oct 31 | ODE Numerical Methods Refresher

Connection to Reading: MIT 18.03 + Evans intro — Euler/RK basics.

Goal

Compare Euler vs RK4 for $\dot{x} = -x + t$ and visualise convergence rates.

Steps

1) **Imports**

```
import numpy as np
import matplotlib.pyplot as plt
```

2) **ODE and solvers**

```
def f(t, x):
    return -x + t

def euler(f, t0, x0, h, N):
    t = np.linspace(t0, t0 + N*h, N+1)
    x = np.zeros(N+1); x[0] = x0
    for i in range(N):
        x[i+1] = x[i] + h*f(t[i], x[i])
    return t, x

def rk4(f, t0, x0, h, N):
    t = np.linspace(t0, t0 + N*h, N+1)
    x = np.zeros(N+1); x[0] = x0
    for i in range(N):
        k1 = f(t[i], x[i])
```

```
        k2 = f(t[i]+h/2, x[i]+h*k1/2)
        k3 = f(t[i]+h/2, x[i]+h*k2/2)
        k4 = f(t[i]+h, x[i]+h*k3)
        x[i+1] = x[i] + (h/6)*(k1+2*k2+2*k3+k4)
    return t, x
```

3) **Convergence experiment**

```
true = lambda t: t - 1 + 2*np.exp(-t)
steps = [0.4, 0.2, 0.1, 0.05]
err_euler, err_rk4 = [], []
for h in steps:
    N = int(2/h)
    t, xe = euler(f, 0, 1, h, N)
    _, xr = rk4(f, 0, 1, h, N)
    err_euler.append(abs(xe[-1] - true(2)))
    err_rk4.append(abs(xr[-1] - true(2)))
plt.loglog(steps, err_euler, 'o-', label='Euler')
plt.loglog(steps, err_rk4, 's--', label='RK4')
plt.loglog(steps, steps, 'k:', label='O(h)')
plt.legend(); plt.xlabel('Δt'); plt.ylabel('Error at t=2'); plt.title('ODE
Solver Convergence'); plt.grid(True, which='both'); plt.show()
```

Expected Outcome: Euler slope ≈ 1; RK4 slope ≈ 4.

🟩Nov 1 – Nov 15 | Brownian Motion Simulation

Connection to Reading: Evans 1–2; Särkkä & Solin 3.1–3.2 — Wiener process properties.

Goal

Simulate 10 Brownian paths and verify $\mathbb{E}[W_t^2] \approx t$.

Steps

```
import numpy as np, matplotlib.pyplot as plt
M, N, T = 10, 1000, 1.0
dt = T/N
# increments and paths
dW = np.sqrt(dt)*np.random.randn(M, N)
W = np.cumsum(dW, axis=1)
W = np.hstack((np.zeros((M,1)), W))
# plots
```

```
time = np.linspace(0, T, N+1)
for i in range(M):
    plt.plot(time, W[i])
plt.title('Brownian Motion Paths'); plt.xlabel('t'); plt.ylabel('W_t');
plt.grid(True); plt.show()
# verify E[W_t^2] ~ t
EW2 = W**2
mean_W2 = EW2.mean(axis=0)
plt.plot(time, mean_W2, label='Empirical E[W_t^2]')
plt.plot(time, time, 'r--', label='Theoretical t')
plt.legend(); plt.xlabel('t'); plt.ylabel('E[W_t^2]'); plt.title('Variance
Check'); plt.grid(True); plt.show()
print(f"At t=1: empirical {mean_W2[-1]:.3f} vs theory 1.000")
```

Expected Outcome: Curves overlap; printed value near 1.

🟩Nov 16 – Nov 30 | Geometric Brownian Motion (GBM)

Connection to Reading: Ito's lemma (Evans 3) and Shreve II 3.1–3.3.

Goal

Simulate GBM and check $\mathbb{E}[S_t] = S_0 e^{\mu t}$.

Steps

```
import numpy as np, matplotlib.pyplot as plt
S0, mu, sigma, T, N, M = 100, 0.05, 0.2, 1.0, 1000, 5000
dt = T/N
Z = np.random.randn(M, N)
W = np.cumsum(np.sqrt(dt)*Z, axis=1)
t = np.linspace(dt, T, N)
S = S0*np.exp((mu-0.5*sigma**2)*t + sigma*W)
S = np.hstack((S0*np.ones((M,1)), S))
emp_mean = S.mean(axis=0)
plt.plot(np.linspace(0,T,N+1), emp_mean, label='Empirical mean')
plt.plot(np.linspace(0,T,N+1), S0*np.exp(mu*np.linspace(0,T,N+1)), 'r--',
label='Theory')
plt.legend(); plt.xlabel('t'); plt.ylabel('E[S_t]'); plt.title('GBM
Expectation'); plt.grid(True); plt.show()
```

Expected Outcome: Empirical mean tracks theory curve.

🟩Dec 1 – Dec 15 | Euler–Maruyama & Milstein for OU

Connection to Reading: Evans 4–5; Øksendal 5–6; Särkkä & Solin Ch.4.

Goal

Compare Euler–Maruyama vs Milstein for OU: $dX_t = -\theta X_t dt + \sigma dW_t$. Plot RMSE vs Δt.

Steps

```
import numpy as np, matplotlib.pyplot as plt
# parameters
theta, sigma, X0, T = 2.0, 0.3, 1.0, 1.0
N_ref = 20000; dt_ref = T/N_ref
# reference path
X_ref = np.zeros(N_ref+1); X_ref[0]=X0
for i in range(N_ref):
    X_ref[i+1] = X_ref[i] - theta*X_ref[i]*dt_ref +
sigma*np.sqrt(dt_ref)*np.random.randn()
X_T_true = X_ref[-1]
# schemes
def euler(dt):
    N = int(T/dt); X = X0
    for _ in range(N):
        X += -theta*X*dt + sigma*np.sqrt(dt)*np.random.randn()
    return X

def milstein(dt):
    # sigma constant -> Milstein == Euler here
    return euler(dt)
# errors for multiple steps
steps = [0.1, 0.05, 0.025, 0.0125]
err_e, err_m = [], []
for dt in steps:
    M=200
    e = [abs(euler(dt)-X_T_true) for _ in range(M)]
    m = [abs(milstein(dt)-X_T_true) for _ in range(M)]
    err_e.append(np.sqrt(np.mean(np.array(e)**2)))
    err_m.append(np.sqrt(np.mean(np.array(m)**2)))
plt.loglog(steps, err_e, 'o-', label='Euler–Maruyama')
plt.loglog(steps, err_m, 's--', label='Milstein (const σ)')
plt.loglog(steps, steps, 'k:', label='O(Δt)')
plt.legend(); plt.xlabel('Δt'); plt.ylabel('RMSE'); plt.title('OU Scheme
Convergence'); plt.grid(True, which='both'); plt.show()
```

Expected Outcome: ≈ first-order slope; Milstein overlaps Euler for constant σ.

🟩Dec 16 – Dec 31 | Monte Carlo Option Pricing

Connection to Reading: Shreve II 4–5 — Black–Scholes.

Goal

Price a European call via Monte Carlo and compare to analytic Black–Scholes.

Steps

```
import numpy as np
from scipy.stats import norm
S0,K,r,sigma,T = 100, 100, 0.05, 0.2, 1.0
M = 100000
Z = np.random.randn(M)
ST = S0*np.exp((r-0.5*sigma**2)*T + sigma*np.sqrt(T)*Z)
price_mc = np.exp(-r*T)*np.mean(np.maximum(ST-K,0))

def bs_call(S0,K,r,sigma,T):
    d1=(np.log(S0/K)+(r+0.5*sigma**2)*T)/(sigma*np.sqrt(T))
    d2=d1-sigma*np.sqrt(T)
    return S0*norm.cdf(d1)-K*np.exp(-r*T)*norm.cdf(d2)
print('Monte Carlo:', price_mc, '  Black-Scholes:', bs_call(S0,K,r,sigma,T))
```

Expected Outcome: MC within ~1% of analytic price.

🟩Jan 1 – Jan 15 | Neural ODE Spiral Demo

Connection to Reading: Chen et al. (2018); Kidger (2022).

Goal

Run `torchdiffeq` spiral to visualise continuous-depth dynamics.

Steps

```
pip install torch torchdiffeq
```

```
import torch, torch.nn as nn
from torchdiffeq import odeint
import matplotlib.pyplot as plt
```

```
class ODEF(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(2,50), nn.Tanh(), nn.Linear(50,2))
    def forward(self, t, y):
        return self.net(y)

func = ODEF()
y0 = torch.tensor([[2., 0.]])
t = torch.linspace(0, 25, 1000)
with torch.no_grad():
    y = odeint(func, y0, t)
plt.plot(y[:,0,0], y[:,0,1])
plt.title('Neural ODE Spiral'); plt.show()
```

Expected Outcome: Smooth spiral trajectory.

🟩Jan 16 – Jan 31 | Neural SDE (NSDE) on Synthetic GBM

Connection to Reading: Li et al. (2020); Särkkä & Solin 5.1–5.3.

Goal

Learn drift/volatility with small MLPs and simulate with `torchsde`.

Steps

```
pip install torch torchsde
```

```
import torch, torch.nn as nn, torchsde
import numpy as np, matplotlib.pyplot as plt

device = 'cuda' if torch.cuda.is_available() else 'cpu'
mu_true, sig_true = 0.08, 0.2
T, N, M = 1.0, 200, 512

# synth GBM data
dt = T/N
Z = torch.randn(M, N, device=device)
W = torch.cumsum(torch.sqrt(torch.tensor(dt, device=device))*Z, dim=1)
t = torch.linspace(0, T, N+1, device=device)
S = torch.exp((mu_true-0.5*sig_true**2)*t[1:] + sig_true*W)
```

```
S = torch.cat([torch.ones(M,1,device=device), S], dim=1)

class Drift(nn.Module):
    def __init__(self): super().__init__(); self.net=nn.Sequential(nn.Linear(2,
16),nn.Tanh(),nn.Linear(16,1))
    def forward(self, t, x): return self.net(torch.cat([x,
torch.ones_like(x)*t], dim=-1))
class Diff(nn.Module):
    def __init__(self): super().__init__(); self.net=nn.Sequential(nn.Linear(2,
16),nn.Tanh(),nn.Linear(16,1), nn.Softplus())
    def forward(self, t, x): return self.net(torch.cat([x,
torch.ones_like(x)*t], dim=-1))
class NSDE(torchsde.SDEIto):
    noise_type='diagonal'; sde_type='ito'
    def __init__(self): super().__init__(); self.F=Drift(); self.G=Diff()
    def f(self,t,y): return self.F(t,y)
    def g(self,t,y): return self.G(t,y)

sde = NSDE().to(device)
opt = torch.optim.Adam(sde.parameters(), 1e-3)

# one-step Gaussian NLL

def loss_one_step(S, dt):
    loss = 0.
    for i in range(S.shape[1]-1):
        St = S[:, i:i+1]
        mean = St + sde.f(t[i], St)*dt
        std = sde.g(t[i], St)*np.sqrt(dt)
        target = S[:, i+1:i+2]
        nll = 0.5*((target-mean)/ (std+1e-6))**2 + torch.log(std+1e-6)
        loss += nll.mean()
    return loss/(S.shape[1]-1)

for epoch in range(50):
    opt.zero_grad(); L = loss_one_step(S, dt); L.backward(); opt.step()
    if epoch%10==0: print(epoch, L.item())

with torch.no_grad():
    y0 = torch.ones(64,1, device=device)
    ts = torch.linspace(0, T, 200, device=device)
    ys = torchsde.sdeint(sde, y0, ts).squeeze(-1).cpu().numpy()
plt.plot(ts.cpu(), ys[:,:10]); plt.title('NSDE sample paths'); plt.show()
```

Expected Outcome: Decreasing loss; realistic stochastic trajectories.

🟩Feb 1 – Feb 15 | Neural CDE on Irregular Synthetic Data

Connection to Reading: Kidger et al. (2020); rough-path intuition.

Goal

Train a Neural CDE with spline-interpolated control on a noisy sine observed at irregular times.

Steps

```
pip install torch torchcde
```

```
import torch, torch.nn as nn, torchcde
import numpy as np, matplotlib.pyplot as plt
T, N = 20.0, 200
base_t = torch.linspace(0, T, N)
x = torch.sin(base_t) + 0.1*torch.randn(N)
mask = torch.rand(N) > 0.3
obs_t, obs_x = base_t[mask], x[mask]
X = torch.stack([obs_t, obs_x], dim=-1)
coeffs = torchcde.natural_cubic_spline_coeffs(X)
X_spline = torchcde.NaturalCubicSpline(coeffs)

class Func(nn.Module):
    def __init__(self, h=32): super().__init__(); self.lin=nn.Linear(h,h);
self.out=nn.Linear(h,h)
    def forward(self,t,z): return torch.tanh(self.out(torch.tanh(self.lin(z))))
class Model(nn.Module):
    def __init__(self, h=32):
        super().__init__(); self.func=Func(h); self.lin_in=nn.Linear(2,h);
self.read=nn.Linear(h,1)
    def forward(self, X):
        z0 = self.lin_in(X.evaluate(X.interval[0]))
        zT = torchcde.cdeint(X=X, func=self.func, z0=z0, t=X._coefficients[...,
0])[-1]
        return self.read(zT)

model = Model(); opt = torch.optim.Adam(model.parameters(), 1e-3)
y_true = x[-1].unsqueeze(0)
for epoch in range(200):
    opt.zero_grad(); yhat = model(X_spline); loss = (yhat.squeeze()-y_true)**2
    loss.backward(); opt.step()
    if epoch%50==0: print(epoch, loss.item())
```

Expected Outcome: Loss decreases; end-to-end NCDE works on irregular inputs.

🟩Feb 16 – Feb 29 | Stochastic Control (Merton Toy)

Connection to Reading: Øksendal 10–11 — HJB/control; toy wealth SDE.

Goal

Compare expected utility under different constant allocations in a simple discrete-time wealth SDE.

Steps

```
import numpy as np, matplotlib.pyplot as plt
T, N, dt = 1.0, 252, 1/252
mu, sigma, r = 0.08, 0.2, 0.02
X0, M = 1.0, 20000
pis = [0.0, 0.3, 0.6]
util = lambda x, g=2.0: x**(1-g)/(1-g)
res = {}
for pi in pis:
    X = np.ones(M)*X0
    for _ in range(N):
        dW = np.sqrt(dt)*np.random.randn(M)
        X += X*(r*dt + pi*((mu-r)*dt + sigma*dW))
    res[pi] = np.mean(util(X))
plt.bar([str(p) for p in pis], [res[p] for p in pis]); plt.ylabel('E[U(X_T)]');
plt.title('Constant Policy Utilities'); plt.show()
```

Expected Outcome: Moderate π often maximises expected utility in this toy setting.

🟩Mar 1 – Mar 15 | FBSDE ↔ PDE Illustration (Heat Equation)

Connection to Reading: Han et al. (2018) — BSDE/PDE link.

Goal

Estimate $u(0,0)$ for heat equation with terminal payoff via backward regression.

Steps

```
import numpy as np
T, N, M = 1.0, 200, 50000
dt = T/N
phi = lambda x: np.maximum(x,0.0)
X = np.zeros((M,N+1))
for i in range(N): X[:,i+1]=X[:,i]+np.sqrt(dt)*np.random.randn(M)
Y = phi(X[:,-1])
for i in reversed(range(N)):
    Xi = X[:,i]
    A = np.vstack([np.ones_like(Xi), Xi, Xi**2]).T
    coef, *_ = np.linalg.lstsq(A, Y, rcond=None)
    Y = (A @ coef)
print('u(0,0) ~', Y.mean())
```

Expected Outcome: Reasonable estimate of $u(0,0)$.

🟩Mar 16 – Mar 31 | Probability Reinforcement (No Code Required)

Focus on proofs/notes; no coding for this block.

🟩Apr 1 – Apr 15 | NODE vs NSDE Comparison

Connection to Reading: Oh (2025) & Kidger (2022) — training and numerics.

Goal

Train NODE and NSDE on the same noisy dynamical signal and compare prediction variance.

Steps (sketch)**

```
# Prepare noisy synthetic series, fit NODE (torchdiffeq) and NSDE (torchsde)
# Plot: NODE point prediction vs NSDE sample path envelope; compute NLL on 1-
step ahead.
```

Expected Outcome: NSDE captures uncertainty envelope better.

🟩Apr 16 – Apr 30 | Literature Synthesis (No Code)

Summaries/memo only.

🟩May 1 – May 15 | OU Mean-Reversion Verification

Connection to Reading: OU mean/variance formulas.

Goal

Empirically verify OU analytical mean and variance curves.

Steps

```
import numpy as np, matplotlib.pyplot as plt
th, sg, X0 = 1.5, 0.4, 1.0
T, N, M = 2.0, 1000, 20000
dt = T/N
X = np.zeros((M,N+1)); X[:,0]=X0
for i in range(N):
    X[:,i+1] = X[:,i] - th*X[:,i]*dt + sg*np.sqrt(dt)*np.random.randn(M)

t = np.linspace(0,T,N+1)
emp_m, emp_v = X.mean(axis=0), X.var(axis=0)
tru_m = X0*np.exp(-th*t)
tru_v = (sg**2)/(2*th)*(1-np.exp(-2*th*t))
plt.plot(t, emp_m, label='Emp mean'); plt.plot(t, tru_m, 'k--', label='Theory
mean'); plt.legend(); plt.show()
plt.plot(t, emp_v, label='Emp var');  plt.plot(t, tru_v, 'k--', label='Theory
var');  plt.legend(); plt.show()
```

Expected Outcome: Close tracking of theory.

🟩May 16 – May 31 | Reproducible Packaging

Goal Create `environment.yml` / `requirements.txt`, add seeding, and ensure notebooks are top-to-bottom runnable.

Steps

```
import random, os, numpy as np, torch

def seed_everything(seed=42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    os.environ['PYTHONHASHSEED']=str(seed)
```

```
    torch.backends.cudnn.deterministic=True
    torch.backends.cudnn.benchmark=False
```

Add a `Makefile` with targets like `make run_all` to execute notebooks.

🟩Jun 1 – Jun 30 | Optional 2D NSDE Prototype

Connection to Reading: Multivariate extensions prepare UROP pitch.

Goal

Model a 2D state (e.g., return & volatility) with diagonal diffusion.

Steps (sketch)

```
# Define drift/diffusion MLPs outputting shape [batch, 2]
# Simulate with torchsde.sdeint; visualise trajectories & covariance over time.
```

Expected Outcome: Qualitative insight into coupled stochastic dynamics.